COLLEGE OF ENGINEERING, ANNA UNIVERSITY,
GUINDY, CHENNAI- 600025

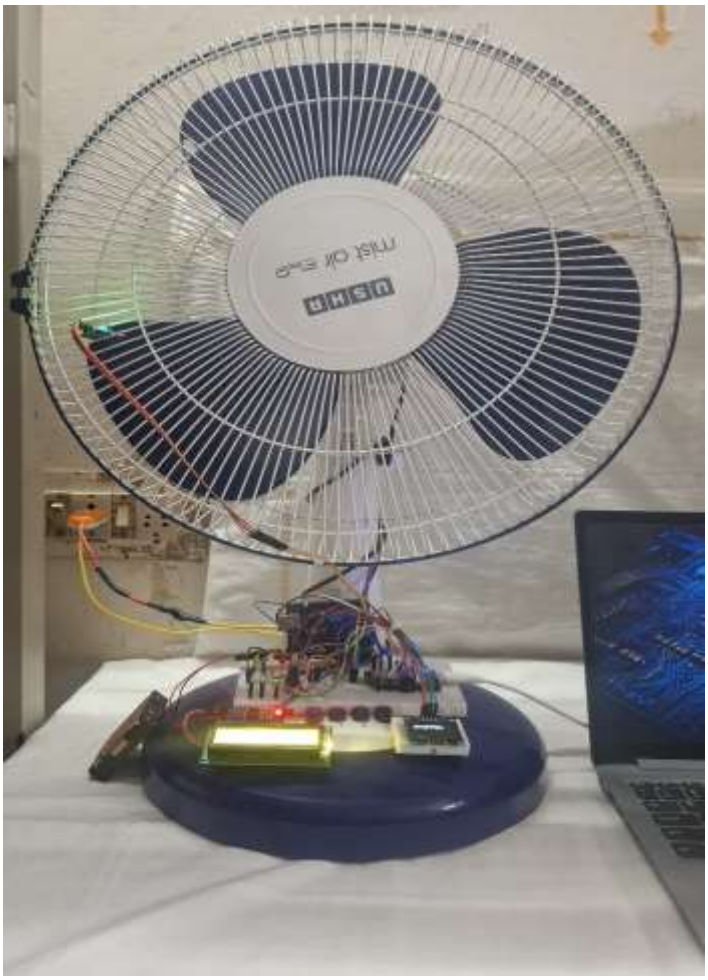


Figure 1: Experimental Setup of Motor Monitoring System

MotorMind AI v1.0: Edge AI-Based Real-Time Motor Health Monitoring and Predictive Fault Detection Using ESP32

AN INTELLIGENT MULTI-SENSOR EMBEDDED SYSTEM FOR PREDICTIVE MAINTENANCE

YUKESH S | B.E Electrical and Electronics Engineering | March 2026
COLLEGE OF ENGINEERING | REG NO: 2024104010

ABSTRACT

This project presents "MotorMind AI v1", an advanced Edge AI-based motor health monitoring and predictive fault detection system developed using the ESP32 microcontroller. The system is designed to detect critical motor faults such as unbalance, overheating, and overcurrent conditions in real time using a combination of multi-sensor data and embedded machine learning.

The system integrates a 3-axis accelerometer (ADXL345) for vibration analysis, an ACS712 current sensor for electrical monitoring, a DS18B20 temperature sensor for thermal tracking, and an IR sensor for RPM measurement. From these sensors, key statistical features such as RMS, mean, and standard deviation are extracted for both vibration and current signals.

A Decision Tree-based machine learning model is deployed directly on the ESP32 using Edge AI techniques, enabling on-device inference without cloud dependency. In addition to fault classification, the system introduces a novel motor health scoring mechanism based on stress analysis of vibration, current, and temperature parameters. This health index is further used to estimate Remaining Useful Life (RUL) and provide predictive maintenance alerts.

The system includes real-time visualization through an OLED display and external LCD, along with LED and buzzer alerts for immediate fault indication. IoT integration using Blynk enables remote monitoring and notifications.

The proposed system offers a low-cost, real-time, and scalable solution for predictive maintenance in industrial environments, reducing downtime and improving operational reliability through intelligent edge processing.

Keywords: *Edge AI, ESP32, Motor Health Monitoring, Predictive Maintenance, Fault Detection, Decision Tree, Vibration Analysis, Current Monitoring, Temperature Monitoring, RMS Features, Embedded Machine Learning, Remaining Useful Life (RUL), Industrial IoT*

TABLE OF CONTENTS

1. Introduction …….3
2. Problem Statement ……..4
3. System Overview
    3.1 Block Diagram ……..7
    3.2 Hardware Description ……...8
        3.2.1 Individual Components ……..9
4. Circuit Diagram
    4.1 Circuit Diagram (Simulation) ……..12
    4.2 Circuit Diagram (Real Time Circuit) ……...13
5. Pin Configuration ……...14
6. System Design
    6.1 System Workflow ……15
    6.2 Sensor Data Acquisition ……...16
    6.3 Signal Processing and Feature Extraction ……17
    6.4 Fault Detection logic (Edge AI) ……...18
    6.5 Output and Communication ……19
7. Results and Discussion ……...20
8. Advantages ……..21
9. Disadvantages ……22
10. Conclusion ……....23
11. Appendix ……..24
12. References ……34

1 INTRODUCTION :

Electric motors are widely used in industrial applications, and their continuous operation is critical for maintaining productivity and system reliability. However, motor failures such as bearing faults, overheating, overcurrent, and mechanical imbalance can lead to unexpected breakdowns, increased maintenance costs, and safety risks.

Traditional motor monitoring systems rely on periodic manual inspection or threshold-based detection methods, which are often inefficient in identifying early-stage faults. These methods fail to capture complex patterns in vibration, current, and temperature signals, making it difficult to predict failures before they occur.

With the advancement of embedded systems and Edge AI, it is now possible to perform real-time condition monitoring and intelligent fault detection directly on low-cost microcontrollers like the ESP32. By integrating multiple sensors and applying machine learning techniques, motor health can be continuously analyzed, enabling predictive maintenance and reducing downtime.

This project proposes an Edge AI-based motor monitoring system that combines multi-sensor data acquisition, feature extraction, and on device machine learning to detect faults and estimate motor health in real time.

2 PROBLEM STATEMENT:

In industrial environments, motor faults often develop gradually and remain undetected until they reach a critical stage. Existing monitoring systems are either expensive, cloud-dependent, or limited to simple threshold-based alerts, which cannot accurately detect early fault conditions.

As shown in the analysis graphs, different fault conditions significantly alter motor behaviour:

- In electrical faults such as overcurrent, the amplitude of the current waveform increases, leading to higher RMS values compared to normal operation.
- Mechanical imbalance introduces periodic fluctuations in motor speed, affecting stability.
- Overheating results in a continuous rise in temperature beyond safe operating limits.
- Bearing faults generate irregular and high-frequency vibration patterns, which are clearly distinguishable in both time-domain and frequency-domain (FFT) analysis.

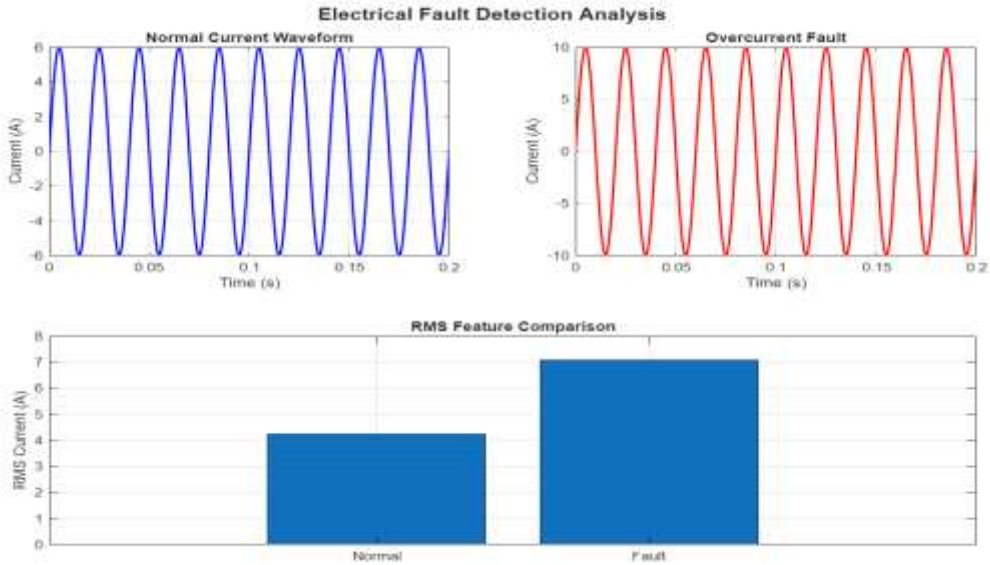
These variations indicate that simple threshold monitoring is insufficient, and a more intelligent system is required to analyse multiple parameters simultaneously.

Therefore, there is a need for a low-cost, real-time, and intelligent motor monitoring system that can:

- Continuously acquire multi-sensor data
- Extract meaningful features such as RMS, mean, and standard deviation
- Detect faults using embedded machine learning
- Predict motor health and provide early maintenance alerts

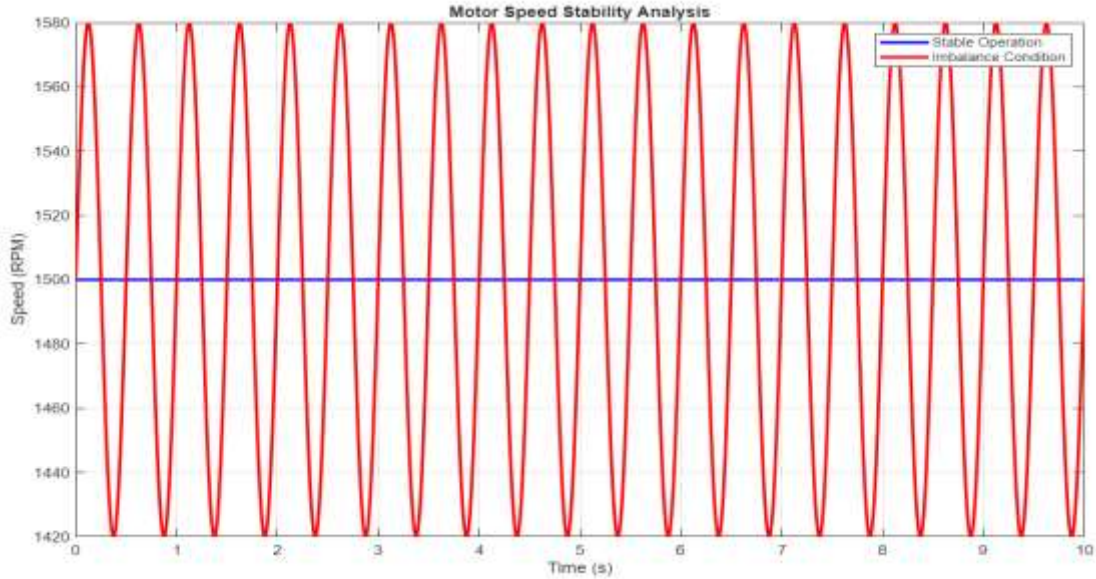
This project addresses these challenges by implementing an Edge AI-based motor fault detection system using ESP32, capable of real-time decision-making without relying on cloud processing.

Figure 2: Electrical Fault Analysis:



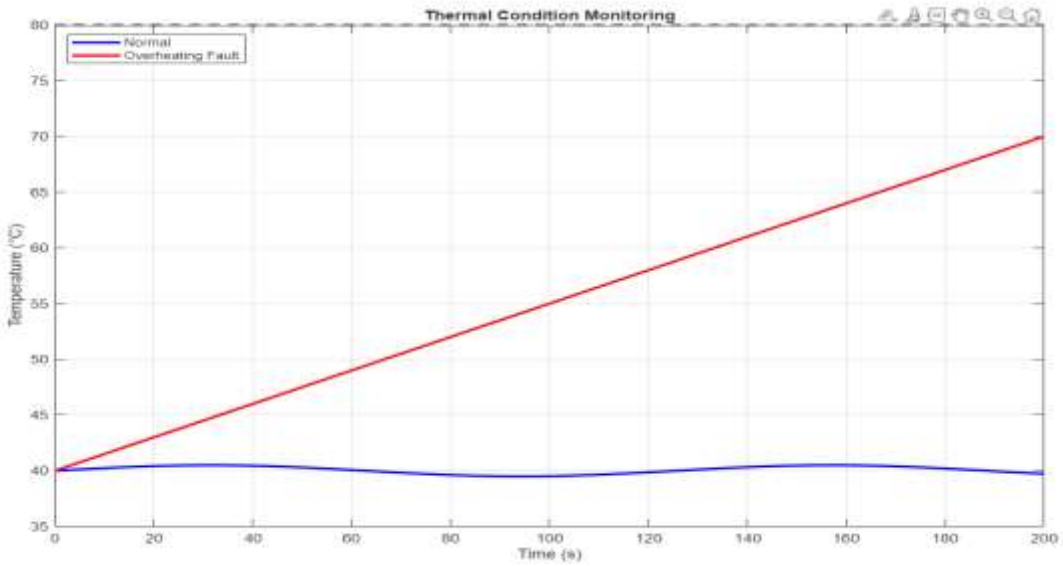
The figure illustrates a significant increase in RMS current under overcurrent fault conditions compared to normal operation, indicating abnormal electrical loading in the motor.

Figure 3: Motor Speed Stability Analysis:



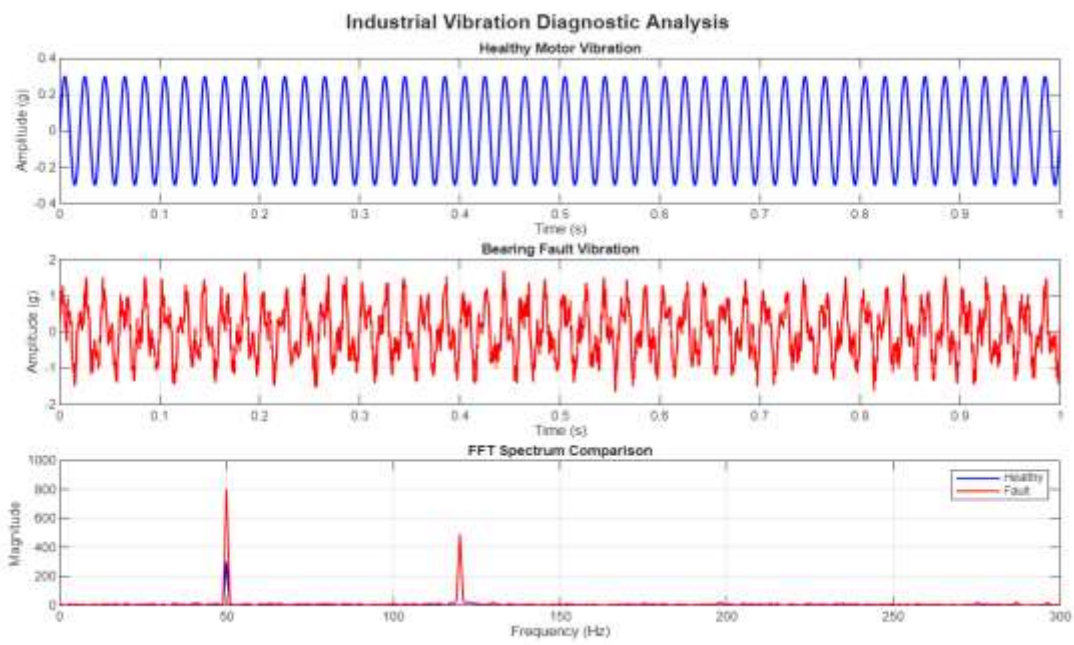
Comparison of stable and imbalance conditions showing periodic fluctuations in speed, indicating mechanical instability.

Figure 4: Thermal Condition Monitoring



Temperature variation under normal and overheating conditions showing continuous rise beyond safe limits.

Figure 5: Vibration & FFT Analysis



Comparison of healthy and faulty vibration signals showing irregular patterns and frequency peaks used for fault detection

3. SYSTEM OVERVIEW:

The system consists of three main sections: sensor layer, processing unit, and output interface.

The motor is continuously monitored using multiple sensors. The ADXL345 accelerometer captures vibration data, the ACS712 sensor measures current consumption, the DS18B20 sensor monitors temperature, and the IR sensor is used for RPM measurement. These sensors provide real-time data representing the electrical, mechanical, and thermal condition of the motor.

All sensor data is fed into the ESP32 microcontroller, which acts as the central processing unit. Inside the ESP32, the system performs data acquisition, followed by signal processing and feature extraction. Key statistical features such as RMS, mean, and standard deviation are computed from the sensor signals.

These extracted features are then passed to a Decision Tree-based machine learning model implemented using Edge AI. The model classifies the motor condition into different states such as normal, unbalance, overheating, and overcurrent.

3.1 BLOCK DIAGRAM

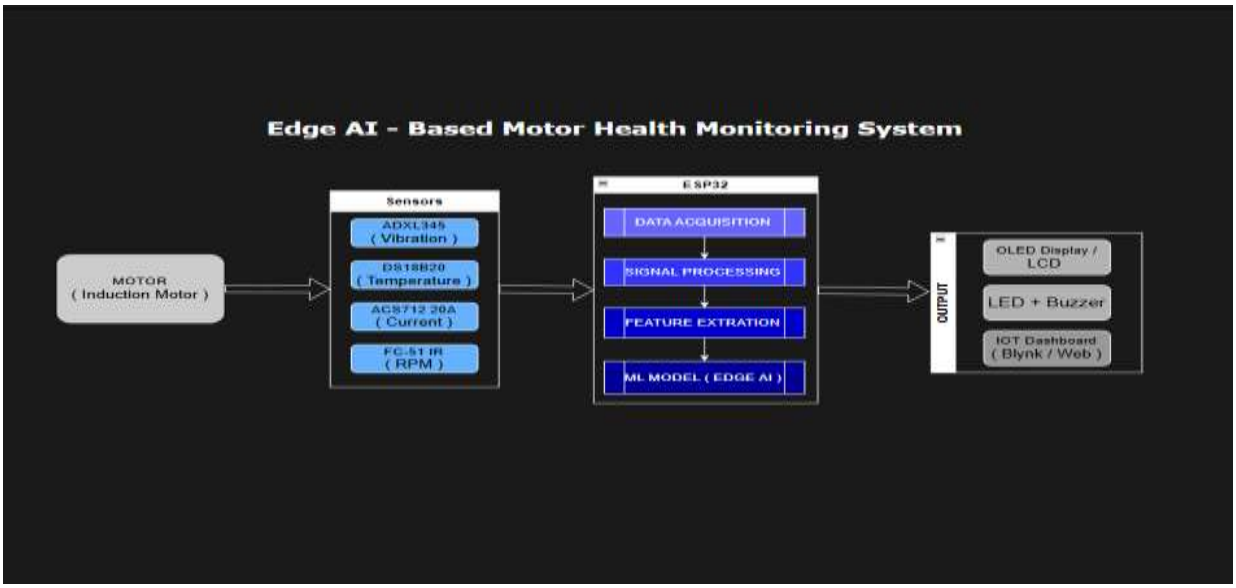


Figure 6: Block Diagram of Edge AI-Based Motor Health Monitoring System

Based on the classification results and computed motor health index, the system generates outputs through multiple interfaces. Real-time information is displayed on an OLED display and an external LCD, while LEDs and a buzzer provide visual and audible alerts. Additionally, IoT integration using Blynk enables remote monitoring and alert notifications.

This architecture ensures complete on-device processing, reducing latency and eliminating dependency on cloud-based systems.

3.2 HARDWARE DESCRIPTION

The hardware implementation of the MotorMind AI v1.0 system is centered around the ESP32 microcontroller, which performs data acquisition, processing, and real-time decision-making.

The ESP32 is selected due to its high processing capability, built-in Wi-Fi support, and suitability for Edge AI applications. It enables on-device machine learning inference and real-time monitoring without requiring external computation.

A set of sensors is used to monitor different aspects of motor behavior. The ADXL345 accelerometer is used to measure vibration signals, which are essential for detecting mechanical faults such as imbalance and bearing defects. The ACS712 current sensor is used to measure motor current, allowing detection of electrical faults such as overcurrent conditions.

The DS18B20 digital temperature sensor is used to monitor motor temperature, which helps in identifying overheating conditions. An IR-based sensor is used to measure motor speed (RPM) by detecting rotational pulses.

For user interaction and alerts, an OLED display is used to show real-time sensor values and motor health status. An external LCD module is used for additional status display. LEDs are used to indicate normal, warning, and fault conditions, while a buzzer provides audible alerts during critical fault situations.

The combination of these components enables comprehensive monitoring of electrical, mechanical, and thermal parameters, forming the foundation for accurate fault detection and predictive analysis.

3.2.1 ESP32 Microcontroller: (7semi ESP32- DEVKIT-IE)

The ESP32 serves as the central processing unit of the system. It performs sensor data acquisition, feature extraction, and machine learning inference. Its built-in Wi-Fi capability enables IoT-based monitoring, while its processing power supports real-time Edge AI execution.

3.2.2 ADXL345 Accelerometer (I2C Interface):

The ADXL345 is a 3-axis digital accelerometer used to measure motor vibration. It provides high-resolution acceleration data, which is used to compute features such as RMS and standard deviation for detecting mechanical faults like imbalance and bearing defects.

3.2.3 ACS712 Current Sensor:

ACS712 20A Current Sensor

The ACS712 sensor is used to measure the motor current. It converts current into an analog voltage, which is read by the ESP32 ADC. The current signal is processed to calculate RMS and statistical features, enabling detection of overcurrent conditions.

3.2.4 DS18B20 Temperature Sensor:

The DS18B20 is a digital temperature sensor used to monitor motor temperature. It provides accurate temperature readings through a one-wire interface, which helps in identifying overheating conditions in the motor.

3.2.5 IR Sensor (RPM Measurement):

An infrared sensor is used to measure motor speed by detecting pulses generated during rotation. The pulse count is used to calculate RPM, which helps in identifying abnormal operating conditions such as imbalance or stoppage.

3.2.6 OLED Display (I2C Interface):

The OLED display is used for real-time visualization of motor parameters such as vibration, current, temperature, RPM, and motor health status. It provides a compact and clear interface for local monitoring.

3.2.7 Arduino (LCD Controller):

An Arduino microcontroller is used as an interface to control the LCD module. The ESP32 sends status messages to the Arduino through UART (Serial2 communication), and the Arduino updates the LCD display accordingly. This separation simplifies display handling and reduces processing load on the ESP32.

3.2.8 LCD Module:

The LCD is used to display system status messages such as connection status, motor condition, and fault alerts. It provides a clear and user-friendly interface for real-time monitoring.

3.2.9 LED Indicators and Buzzer:

LEDs are used to indicate different motor states such as normal, warning, and fault conditions. A buzzer is used to provide audible alerts during critical fault situations, ensuring immediate user attention.

3.2.10 Power Supply Unit:

In the implemented system, the ESP32 is powered through a USB connection from a laptop, which provides a stable 5V supply during development and testing. The Arduino used for LCD control is powered separately using a 6V battery source. This dual power arrangement ensures independent operation of both controllers and avoids interference between the processing unit and display subsystem.

3.2.11 Capacitors (Decoupling and Filtering):

Capacitors are used in the circuit to stabilize the power supply and reduce noise. Decoupling capacitors are placed near the ESP32 and sensor modules to suppress voltage fluctuations. This improves the reliability of the system

and ensures accurate sensor readings, especially for analog signals such as current measurement.

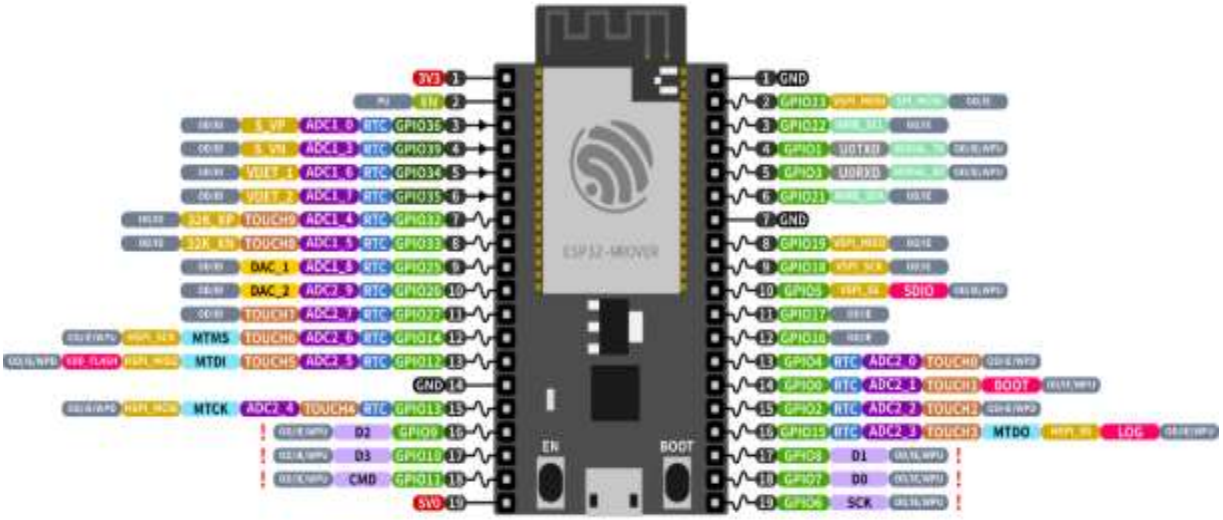
3.2.12 Breadboard (Prototyping Platform):

A breadboard is used for assembling and testing the circuit without soldering. It allows easy modification of connections during development and debugging, making it suitable for prototyping the system.

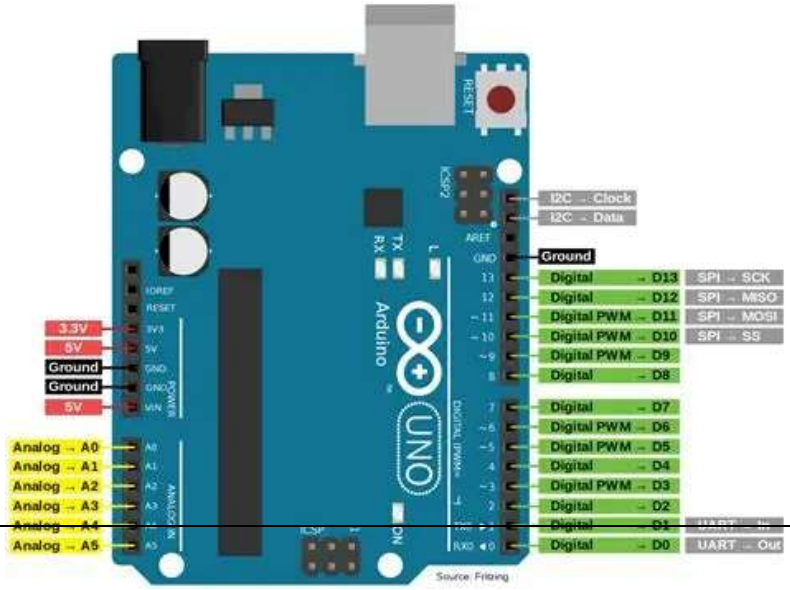
3.2.13 Connecting Wires (Jumper Wires):

Jumper wires are used to connect the ESP32, sensors, Arduino, and other components. Proper wiring ensures stable communication and reliable system operation.

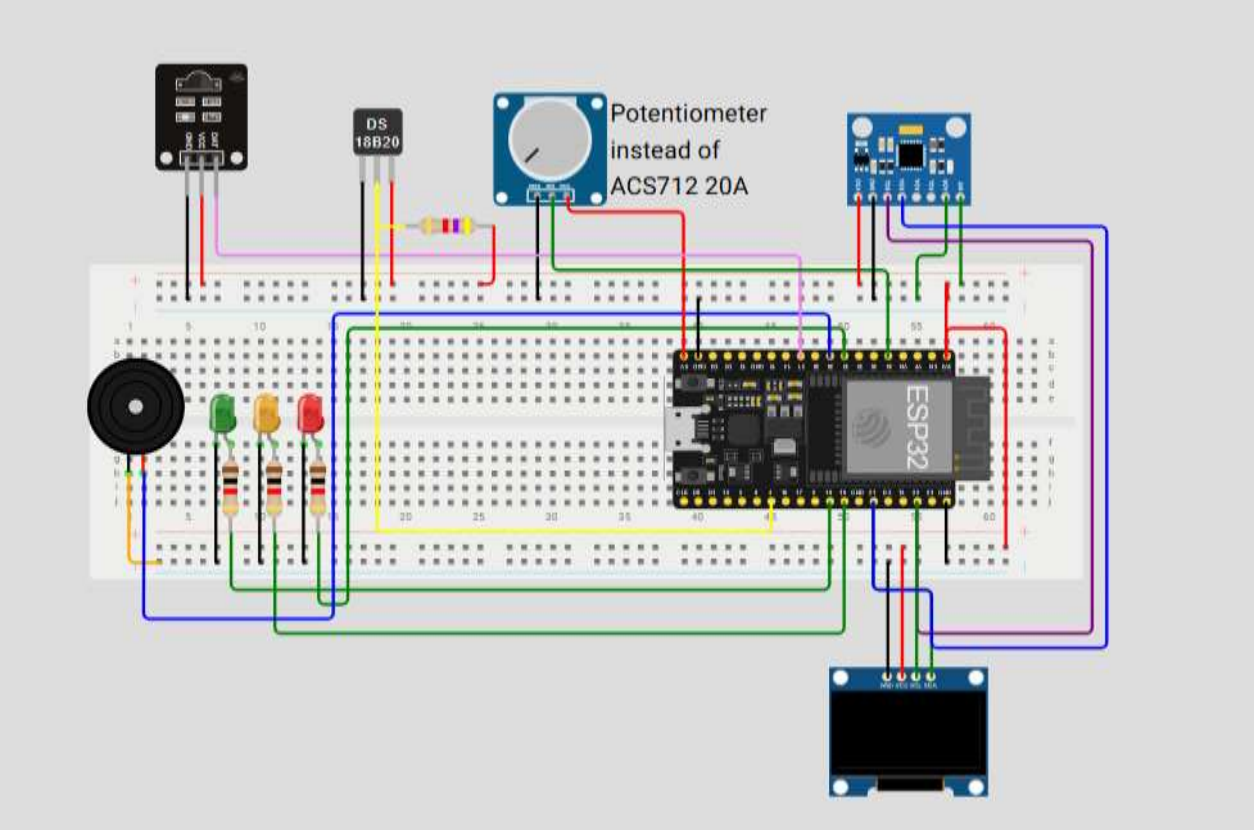
3.2.14 ESP32 PINOUT:

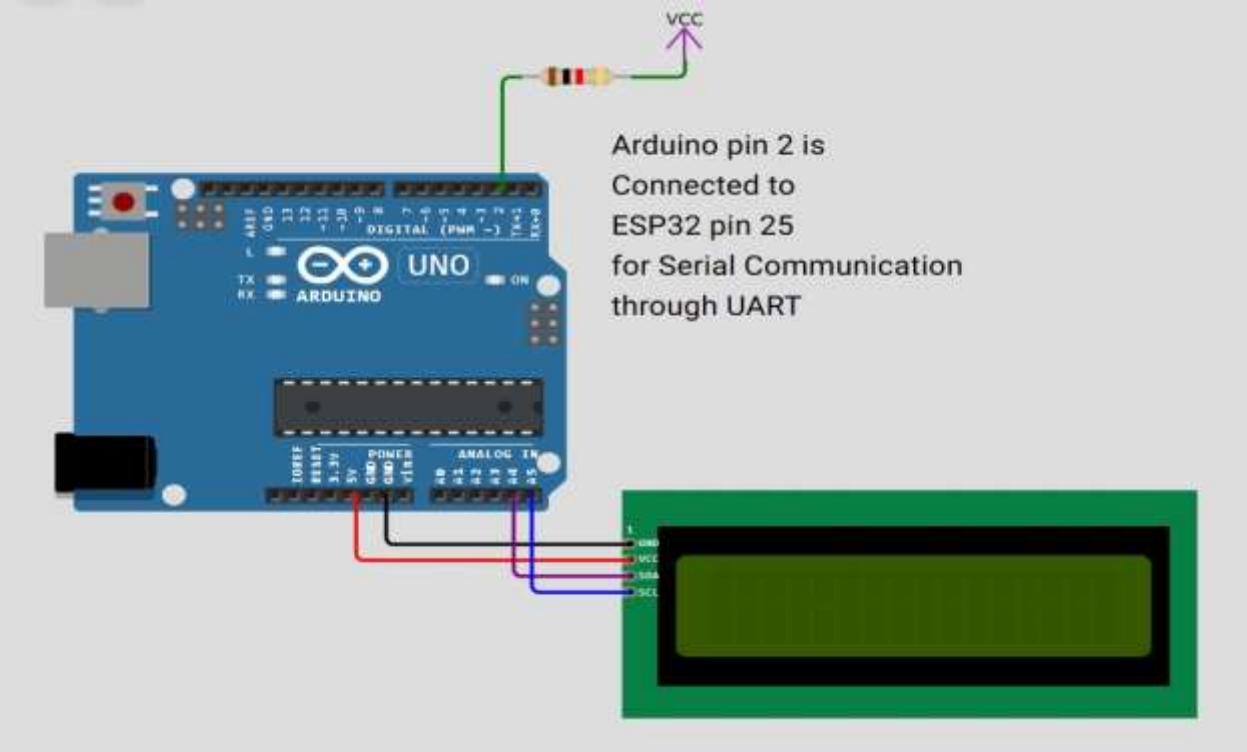


3.2.15 ARDUINO UNO PINOUT:

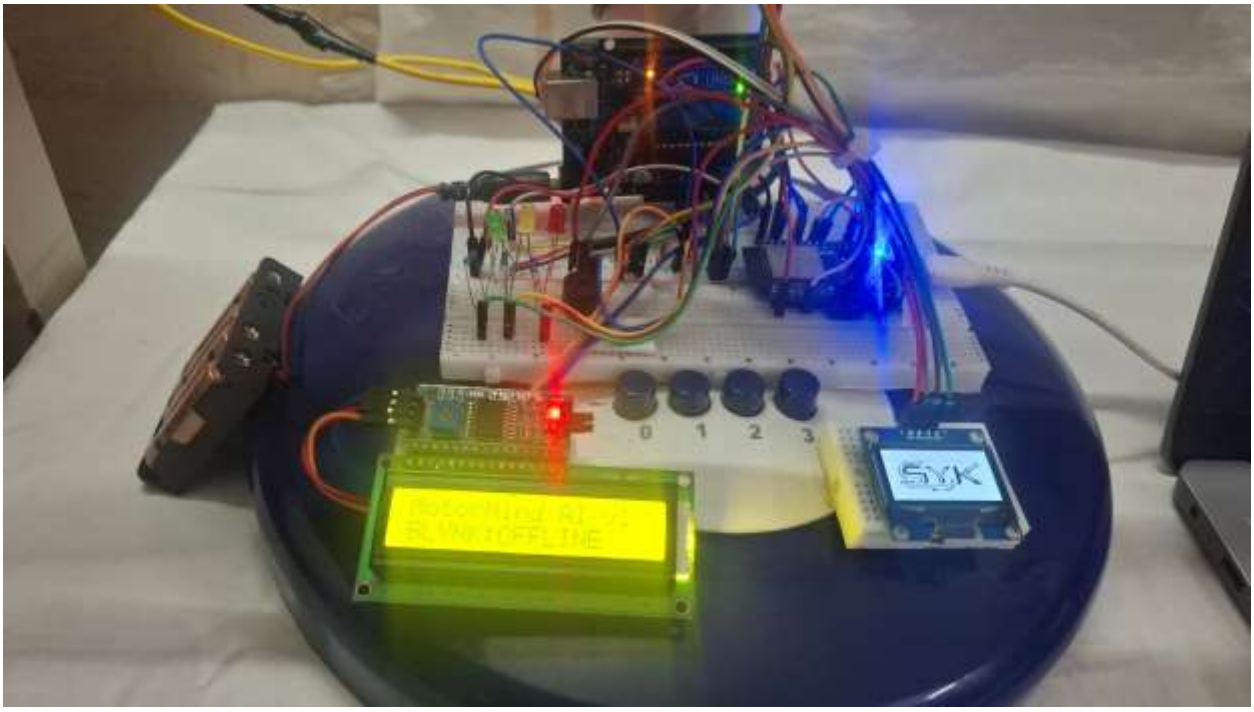


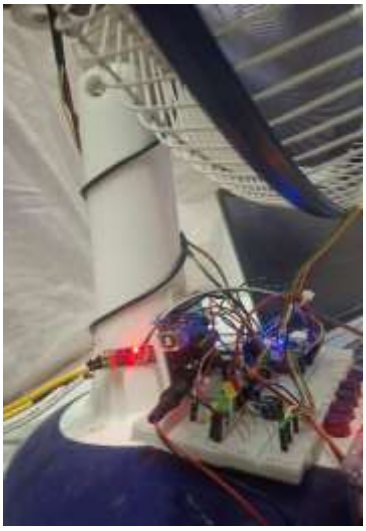
4.1 CIRCUIT DIAGRAM(SIMULATION)





4.2 CIRCUIT DIAGRAM (REAL TIME CIRCUIT)





5. PIN CONFIGURATION:

COMPONENTS	ESP32 PIN	DESCRIPTION
IR Sensor (Speed)	GPIO27	RPM measurement using pulse detection
Current Sensor (ACS712)	GPIO34	Analog input for current measurement
Temperature Sensor (DS18B20)	GPIO4	One-wire digital temperature sensing
Buzzer	GPIO26	Fault alert (sound indication)
LED (Normal)	GPIO18	Indicates normal operation
LED (Warning)	GPIO19	Indicates warning condition
LED (Fault)	GPIO33	Indicates fault condition
OLED Display (SDA)	GPIO21	I2C data line
OLED Display (SCL)	GPIO22	I2C clock line
ADXL345 (SDA)	GPIO21	I2C shared with OLED
ADXL345 (SCL)	GPIO22	I2C shared with OLED
UART TX (to Arduino)	GPIO25	Serial data transmission
UART RX (from ESP32)	**D2**(Arduino)	SoftwareSerial data reception
LCD (SDA)	**A4**(Arduino)	I2C data line
LCD (SCL)	**A5**(Arduino)	I2C clock line

Note: The I2C protocol is used to interface multiple devices (OLED display and ADXL345 accelerometer) using shared SDA and SCL lines, improving hardware efficiency and reducing pin usage and A common ground is maintained between ESP32 and Arduino to ensure proper UART communication and signal reference.

6 SOFTWARE DESIGN

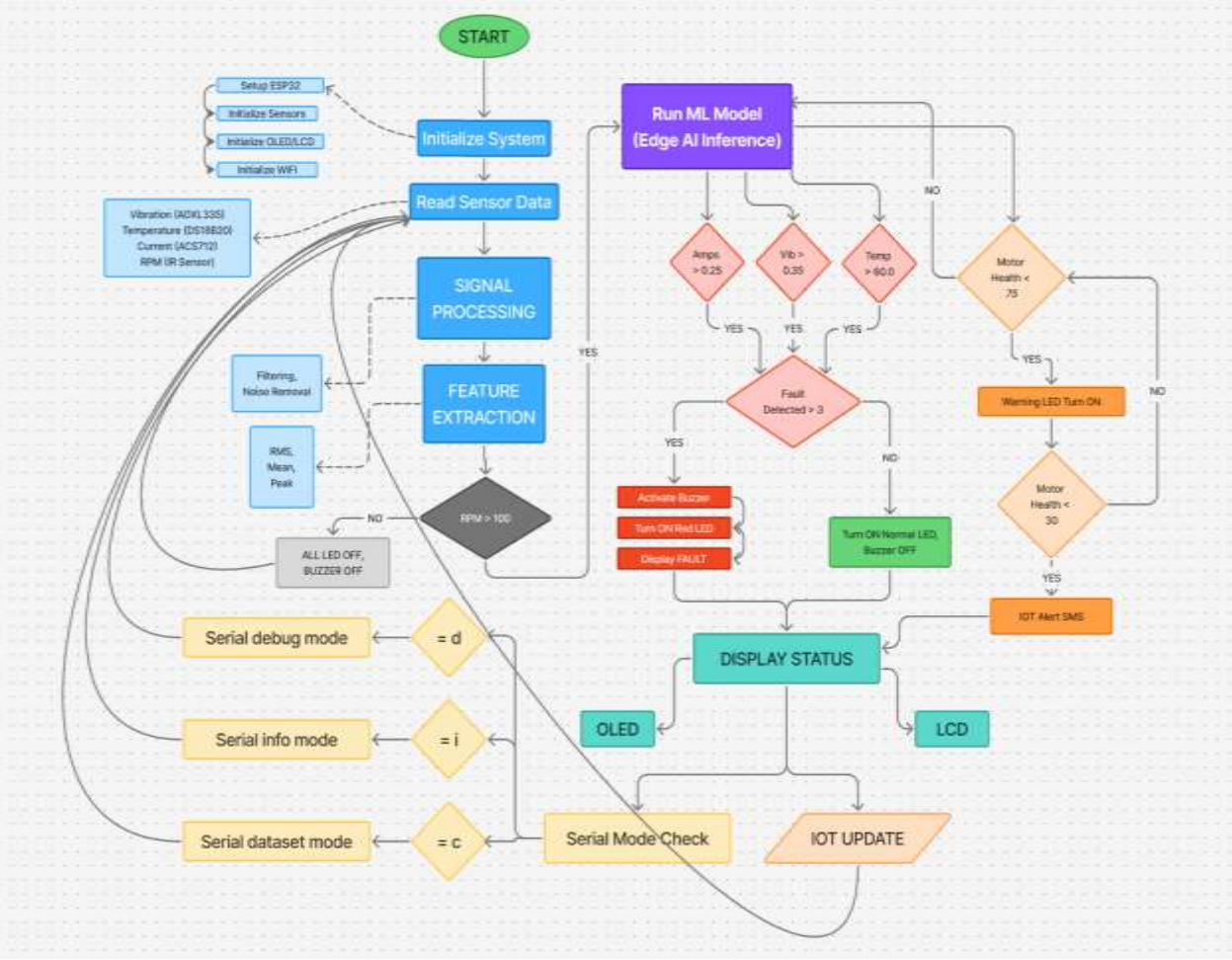
The software design of the proposed MotorMind AI system is responsible for real-time data acquisition, processing, fault detection, and user notification. The system is implemented using embedded C/C++ on the ESP32 platform, integrating sensor interfacing, signal processing, and decision-making algorithms.

The software architecture is designed to ensure efficient execution of multiple tasks, including sensor data collection, feature extraction, and fault classification. The processed results are used to determine the health condition of the motor and trigger appropriate alerts through display and notification systems.

6.1 SYSTEM WORKFLOW

The system operates in a continuous loop where sensor data is collected, processed, and analysed to determine the motor condition. Initially, the ESP32 initializes all sensors, communication interfaces, and output devices.

Figure 7: Software Flowchart of MotorMind AI System



During operation, real-time data from vibration, temperature, current, and speed sensors is acquired. The collected data is then processed to extract meaningful features such as RMS values and threshold comparisons.

Based on these processed values, the system classifies the motor condition into normal, warning, or fault states.

6.2 SENSOR DATA ACQUISITION:

The sensor data acquisition module is responsible for collecting real-time data from multiple sensors interfaced with the ESP32. The system utilizes a combination of analog and digital communication protocols to ensure accurate and efficient data collection.

The vibration data is obtained using the ADXL345 accelerometer, which communicates with the ESP32 through the I2C protocol. The sensor provides acceleration values that are used to analyze motor vibration characteristics.

The temperature data is measured using the DS18B20 digital temperature sensor, which operates on a one-wire communication interface. This sensor provides precise temperature readings, enabling effective monitoring of thermal conditions.

The current measurement is performed using the ACS712 sensor. During prototype testing, a potentiometer is used to simulate varying current conditions. The analog output from the sensor is read using the ESP32 ADC (Analog-to-Digital Converter) pin, allowing the system to monitor electrical load variations.

The rotational speed of the motor is measured using an IR sensor, which detects pulses generated by the rotating object. The ESP32 counts these pulses over time to calculate the RPM (Revolutions Per Minute) of the motor.

All sensor data is continuously sampled and updated in real time, ensuring accurate monitoring of motor conditions for further processing and fault detection.

The use of multiple sensor types (analog, digital, and I2C-based) ensures robust data acquisition and enables comprehensive monitoring of electrical, mechanical, and thermal parameters.

6.3 SIGNAL PROCESSING AND FEATURE EXTRACTION

The acquired sensor data is processed to extract meaningful features that can be used for fault detection. Raw sensor signals often contain noise and variations, making direct analysis ineffective. Therefore, statistical feature extraction techniques are applied to obtain reliable indicators of motor condition.

For vibration and current signals, multiple samples are collected over a fixed interval. From these samples, key statistical features such as Root Mean Square (RMS), mean, and standard deviation are computed. The RMS value represents the effective magnitude of the signal, while the standard deviation indicates variations and irregularities in the signal.

These features help in distinguishing between normal and faulty operating conditions. For example, an increase in RMS value may indicate abnormal vibration or excessive current draw, while higher standard deviation reflects instability in the system.

Temperature data is directly monitored and compared against predefined thresholds to identify overheating conditions. Similarly, RPM values are analyzed to detect speed fluctuations and abnormal motor behavior.

The extracted features are then used as inputs for the fault detection algorithm, enabling accurate classification of motor conditions.

6.4 FAULT DETECTION LOGIC (EDGE AI)

The fault detection mechanism in the MotorMind AI system is implemented using an Edge AI approach, where all processing and decision-making are performed directly on the ESP32 microcontroller. This eliminates the need for cloud-based computation and enables real-time fault detection with low latency.

The system utilizes a Decision Tree-based model to classify motor conditions based on extracted features such as RMS, mean, and standard deviation of vibration and current signals, along with temperature and RPM data. These features are continuously evaluated to determine the operating condition of the motor.

Based on the processed inputs, the system classifies motor states into categories such as normal operation, warning, and fault conditions. Specific fault types such as overcurrent, overheating, and imbalance are identified by analysing deviations in sensor parameters from predefined thresholds and patterns.

In addition to fault classification, a motor health index is calculated based on the combined effect of vibration, current, and temperature parameters. This health index provides a quantitative measure of the motor condition. Using this value, the system estimates the Remaining Useful Life (RUL) and generates predictive maintenance alerts when the motor condition deteriorates.

To improve reliability, the system incorporates a confirmation mechanism where abnormal conditions must persist for a defined number of cycles before being classified as a fault. This reduces false positives and ensures stable operation.

The implementation of on-device machine learning combined with multi-parameter health evaluation makes the system more efficient and reliable compared to traditional threshold-based monitoring systems.

6.5 OUTPUT AND COMMUNICATION

The system provides real-time output and alerts based on the detected motor condition. Multiple output interfaces are used to ensure effective user interaction and monitoring.

The OLED display connected to the ESP32 is used to show real-time sensor values such as vibration, current, temperature, RPM, and motor health status. This provides immediate local feedback to the user.

An external LCD module is controlled using an Arduino microcontroller. The ESP32 communicates with the Arduino through UART (serial communication), sending processed data and status messages. The Arduino updates the LCD display accordingly, providing an additional interface for system monitoring.

Visual indication of motor status is achieved using LEDs. Different LEDs are used to represent normal, warning, and fault conditions. In addition, a buzzer is activated during critical fault conditions to provide an audible alert, ensuring immediate user attention.

The system also supports IoT-based monitoring using the Blynk platform. Sensor data and motor status can be transmitted wirelessly, allowing remote monitoring and alert notifications.

This multi-level output and communication design ensures that the user is continuously informed about the motor condition both locally and remotely.

The combination of local and remote monitoring enhances system usability and ensures timely response to potential motor faults.

Note: The complete source code for the system is provided in Appendix A.

7. RESULTS AND DISCUSSION:

The developed MotorMind AI system was tested under different operating conditions to evaluate its performance in real-time motor health monitoring and fault detection. The system successfully acquired data from multiple sensors and processed it using embedded algorithms on the ESP32.

The results demonstrate that the system can effectively detect variations in vibration, current, temperature, and speed, and accurately classify motor conditions into normal, warning, and fault states. The use of feature extraction techniques such as RMS and statistical analysis enabled reliable identification of abnormal conditions.

The system provided real-time feedback through the OLED display, LCD module, LEDs, and buzzer alerts. Additionally, IoT-based monitoring allowed remote observation of motor parameters.

Overall, the system performed efficiently in detecting faults such as overcurrent, overheating, and imbalance, validating the effectiveness of the proposed Edge AI-based approach.

Figure: Real-time OLED display showing motor parameters, LCD displaying system status and IOT Dashboard

The experimental results confirm that the MotorMind AI system is capable of accurately monitoring motor parameters in real time. The OLED display shows instantaneous sensor readings and system status, while the LCD module provides a clear indication of motor condition such as “NORMAL,” ensuring easy local monitoring.

The IoT dashboard further demonstrates the system’s ability to transmit data wirelessly and visualize parameters such as current, temperature, and health index. The observed values indicate stable operation under normal conditions, with consistent readings across all monitored parameters.

During testing, the system responded effectively to changes in input conditions, and the fault detection logic successfully classified the motor state based on sensor data. The integration of multiple sensors enabled comprehensive monitoring, improving reliability compared to single-parameter systems.

Overall, the results validate that the proposed Edge AI-based system can perform real-time motor health monitoring with accurate fault detection, efficient visualization, and reliable communication.

8. ADVANTAGES:

The proposed MotorMind AI system offers efficient real-time motor health monitoring using an Edge AI approach. All processing is performed directly on the ESP32, eliminating the need for cloud dependency and ensuring low-latency fault detection.

A key innovation of the system is the integration of multi-sensor data (vibration, temperature, and current) with on-device decision-making. This enables accurate detection of faults such as imbalance, overheating, and overcurrent using a single compact embedded platform.

The system provides multi-level output through OLED display, LCD module, LEDs, and buzzer, ensuring both local and remote monitoring. Additionally, IoT integration allows real-time visualization of motor parameters.

The modular design using ESP32 and Arduino improves system flexibility, where processing and display operations are handled independently, enhancing overall system efficiency.

9. LIMITATIONS:

During the development and testing of the prototype, several practical limitations were observed. The breadboard-based implementation resulted in loose jumper wire connections, which occasionally caused unstable sensor readings and required manual adjustment.

It was observed that the system required a common and stable power source for proper operation. When different power sources were used for the ESP32 and Arduino, the system did not turn off completely and the display remained dim, indicating improper power synchronization.

While testing display configurations, it was found that using an LCD directly with the ESP32 affected the performance of the OLED display. In such cases, the OLED display became unresponsive or stuck, which influenced the decision to use a separate Arduino for LCD control.

The vibration analysis was performed without advanced signal processing techniques such as FFT, which restricted the system's ability to detect complex fault patterns.

These observations highlight the practical challenges encountered during hardware implementation and provide scope for future improvements in system stability and performance.

10. CONCLUSION:

The MotorMind AI system successfully demonstrates a real-time motor health monitoring and fault detection solution using an Edge AI approach. The system integrates multiple sensors to monitor vibration, temperature, current, and speed parameters, enabling comprehensive analysis of motor conditions.

The implementation of on-device processing using the ESP32 ensures low-latency decision-making and eliminates dependency on cloud-based

systems. The system was able to accurately classify motor conditions into normal, warning, and fault states based on processed sensor data.

The use of multiple output interfaces, including OLED display, LCD module, LEDs, buzzer, and IoT dashboard, provided effective real-time monitoring and user interaction. The system also demonstrated stable performance during testing and successfully responded to changes in operating conditions.

Overall, the proposed system offers a cost-effective, scalable, and efficient solution for predictive maintenance in motor monitoring applications. The project highlights the potential of combining embedded systems with Edge AI for real-time industrial applications.

Future improvements can focus on enhancing model accuracy, improving hardware robustness, and integrating advanced machine learning techniques for better fault prediction.

APPENDIX A:

SOURCE CODE

The main firmware code used for implementing the MotorMind AI system is included in this appendix for reference. Due to the modular structure of the project, additional supporting files such as configuration headers, display modules, and auxiliary functions are not included in this document.

The complete project source code, including all modules and updates, is available in the GitHub repository for detailed reference and future development.

GitHub Repository: https://github.com/YUKESH0620/MotorMind-AI

```
/*
=====================================================
MotorMind AI v1
Edge AI Motor Health Monitoring System
Author: Yukesh

Main Firmware
=====================================================
*/
#include <Wire.h>
#include <WiFi.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_ADXL345_U.h>
#include <DallasTemperature.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#include "config.h"
#include "sensors.h"
#include "features.h"
#include "ai_engine.h"
#include "health_model.h"
#include "display_oled.h"

#include "iot_blynk.h"
#include "alerts.h"
```

```
#include "motor_model.h"
#include "logger.h"
#include "calibration.h"

// ================= GLOBAL OBJECTS =================
Adafruit_ADXL345_Unified accel = Adafruit_ADXL345_Unified(12345);
OneWire oneWire(ONE_WIRE_BUS);
DallasTemperature tempSensor(&oneWire);
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

Eloquent::ML::Port::DecisionTree model;

// ================= GLOBAL VARIABLES =================
volatile unsigned long pulseCount = 0;
volatile unsigned long lastPulseTime = 0;

float filteredRPM = 0;
float previousHealth = 100;
float offsetVoltage = ACS712_OFFSET;

int abnormalCount = 0;
String fault = "NORMAL";

BlynkTimer timer;

float temperature = 0;
```

```
float vibration = 0;
float current = 0;
int motorhealth = 0;
int Rpm = 0;
int RUL = 100;

int normalLED = 0;
int warningLED = 0;
int faultLED = 0;
int buzzer = 0;

String maintenance;
bool alertSent = false;

int logMode = LOG_DEBUG;

// ================= SENSOR DATA ==================
float vibMean, vibRMS, vibStdDev;
float currentMean, currentRMS, currentStdDev;

float rpm = 0;
float tempC = 0;

float motorHealth = 100;
int aiPrediction = 0;
```

```
float features[8];
MotorFeatures f;

unsigned long rpmTimer = 0;
const int RPM_INTERVAL = 1000;


// ================= SETUP =================
void setup()
{
    Serial.begin(115200);
    Serial2.begin(9600, SERIAL_8N1, -1, 25);

    Wire.begin(SDA_PIN, SCL_PIN);

    pinMode(LED_NORMAL, OUTPUT);
    pinMode(LED_WARNING, OUTPUT);
    pinMode(LED_FAULT, OUTPUT);
    pinMode(BUZZER_PIN, OUTPUT);

    pinMode(IR_PIN, INPUT);
    attachInterrupt(digitalPinToInterrupt(IR_PIN), countPulse, RISING);

    analogReadResolution(12);
    analogSetPinAttenuation(CURRENT_PIN, ADC_11db);
```

```
  // WiFi
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  delay(2000);
  manageWiFi();

  // Blynk timer
  timer.setInterval(2000L, sendDataToBlynk);

  // Sensors
  accel.begin();
  accel.setRange(ADXL345_RANGE_2_G);
  tempSensor.begin();

  // ACS712 Calibration
  analogReadResolution(12);
  analogSetPinAttenuation(CURRENT_PIN, ADC_11db);
  calibrateCurrentSensor();

  // OLED
  display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDRESS);

  showStartupScreen();
}

// ================= MAIN LOOP =================
void loop()
```

```
{

  if (wifiConnected)
  {
    Blynk.run();
  }
  timer.run();

  // ================= SENSOR TASK =================

      readVibration(vibMean, vibRMS, vibStdDev);
      readCurrent(currentMean, currentRMS, currentStdDev);

      rpm = readRPM();
      tempC = readTemperature();

  // ================= AI TASK =================

      f.vibRMS = vibRMS;
      f.vibMean = vibMean;
      f.vibStdDev = vibStdDev;

      f.currentRMS = currentRMS;
      f.currentMean = currentMean;
```

```
    f.currentStdDev = currentStdDev;

    f.rpm = rpm;
    f.temperature = tempC;

    buildFeatureArray(f, features);

    // ===== AI INFERENCE =====
    aiPrediction = runAI(features);
    fault = updateFaultState(aiPrediction, rpm);

    // ===== MOTOR HEALTH =====
    motorHealth = computeMotorHealth(vibStdDev, currentRMS, tempC);

    RUL = estimateRUL(motorHealth);
    maintenance = maintenanceAdvice(RUL);

// ================= DISPLAY TASK =================

    bool warningState = checkWarningState(motorHealth);
    updateAlerts(fault, warningState);

    // ===== OLED =====
   if(fault == "STOPPED")
       showStoppedScreen();
   else
```

```
    displayMotorData(vibRMS, currentRMS, rpm, tempC, motorHealth, fault);

// ===== DATA FOR IOT =====
temperature = tempC;
vibration = vibRMS;
current = currentRMS;
Rpm = rpm;
motorhealth = motorHealth;

// ===== ALERT NOTIFICATION =====
checkMotorAlert(motorHealth);

// ======LCD Through Arduino UNO======
if(motorHealth < 30 )
    sendLCD("MAINTENANCE!");

else if(fault == "UNBALANCE")
    sendLCD("FAULT:UNBALANCE");

else if(fault == "OVERCURRENT")
    sendLCD("FAULT:OVERCURRENT");

else if(fault == "OVERHEAT")
    sendLCD("FAULT:OVERHEAT");

else if(fault == "STOPPED")
```

```
        sendLCD("STATUS:STOPPED");

    else
        sendLCD("STATUS:NORMAL");

// ================= LOGGER TASK =================

    runLogger(
        vibRMS,
        vibMean,
        vibStdDev,
        currentRMS,
        currentMean,
        currentStdDev,
        rpm,
        tempC,
        motorHealth,
        aiPrediction,
        features,
        fault
    );
}
```

12. REFERENCE:

1) Unbalance Detection in Induction Motors through Vibration Signals Using Texture Features(https://www.mdpi.com/2076-3417/13/10/6137)

2) Bearing Fault Detection Vibration Analysis - How To Measure Vibration Frequency (https://ncd.io/blog/bearing-fault-detection-vibration-analysis/)

3) ijarsct.co.in (https://ijarsct.co.in/Paper24141.pdf)

4) ADXL335 (Rev. B) https://www.analog.com/media/en/technical-documentation/data-sheets/adxl335.pd

5) GPIO & RTC GPIO - ESP32 - — ESP-IDF Programming Guide v5.5.3 documentation https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/gpio.html

6) Analog to Digital Converter (ADC) - ESP32 - — ESP-IDF Programming Guide v4.4 documentation https://docs.espressif.com/projects/esp-idf/en/v4.4/esp32/api-reference/peripherals/adc.html

7) ESP32 UART - Serial Communication, Send and Receive Data (Arduino IDE) | Random Nerd Tutorials https://randomnerdtutorials.com/esp32-uart-communication-serial-arduino/

8) Digital Filtering in Vibration Test Control - Vibration Research https://vibrationresearch.com/blog/digital-filtering-in-vibration-test-control/

9) Embedded ML: How to Run Machine Learning Models on Microcontrollers https://promwad.com/news/embedded-ml-microcontrollers-model-deployment

10) Easy TinyML on ESP32 and Arduino - Hackster.io https://www.hackster.io/news/easy-tinyml-on-esp32-and-arduino-a9dbc509f26c

11) TensorFlow Lite On ESP32 – OpenELAB Technology Ltd. https://openelab.io/blogs/learn/tensorflow-lite-on-esp32?srsltid=AfmBOoqRnDycnXyX2mZ5-STp6OKQ7RoatqsWYSR_W_QToV55sjV_R034